

Lecture: Relational algebra and SQL

SQL is the surface. Relational algebra is the reason the surface works.

A relation is a set of tuples with named attributes. Operators take relations and return relations. If your query cannot be sketched with these operators, it is probably doing something procedural that belongs in the application.

Core operators

- **Select (σ)** — keep rows that match a predicate.
- **Project (π)** — keep columns, drop duplicates in the algebra (SQL `SELECT DISTINCT`).
- **Cross product ($\times$)** — every pair of rows. Rarely written by hand; joins are the grown-up form.
- **Join ($\bowtie$)** — a product plus a select, usually on a foreign key.
- **Union, difference** — same schema required.
- **Aggregation** — not in the original algebra; in SQL this is `GROUP BY`.

When you write `FROM a JOIN b ON a.id = b.a_id`, you are doing a theta-join. When you forget the `ON` clause, you are doing a cross product and then wondering why the result has 4 million rows.

SELECT that we will actually use

```
SELECT s.name, COUNT(*) AS n
FROM enrolment e
JOIN student s ON s.id = e.student_id
WHERE e.year = 2026
GROUP BY s.name
HAVING COUNT(*) >= 2
ORDER BY n DESC;
```

Read it in this order, not top to bottom: `FROM/JOIN` (the relations), `WHERE` (σ), `GROUP BY`, `HAVING`, `SELECT` (π plus aggregates), `ORDER BY` (not relational).

Subqueries belong in `WHERE` when you mean “exists a row”, and in `FROM` when you need a named intermediate relation. Correlated subqueries are allowed. They are also how people accidentally write $O(n^2)$ SQL.

Integrity

Primary keys, foreign keys, and `NOT NULL` are not bureaucracy. They are how the database refuses to store a fact that cannot be true. If your schema needs a comment “please do not

insert orphans”, you wanted a foreign key.